

FoFa – Technical Design Document

Version: 1.0

Date: 2026-04-15

Audience: Engineering, Product, Design, QA

Status: Baseline

Table of Contents

1. [Overview](#)
 2. [Goals and Non-Goals](#)
 3. [System Architecture](#)
 4. [Data Model](#)
 5. [API Design](#)
 6. [Frontend Architecture](#)
 7. [Authentication and Authorization](#)
 8. [File Uploads and Media](#)
 9. [Email Service](#)
 10. [Testing Strategy](#)
 11. [Deployment](#)
 12. [Security Considerations](#)
 13. [Known Limitations and Future Work](#)
-

1. Overview

FoFa (Foster Families) is a community platform for foster families. It provides a shared space where verified members can post announcements, exchange direct messages, manage family profiles, discover other community members, and coordinate playdates.

The platform is built as a monorepo containing an Express REST API, a React single-page application, a React Native mobile app, and a Cypress end-to-end test suite.

Core Feature Set

Feature	Description
Auth	Email-verified registration, JWT-based sessions, password reset
Announcements feed	Create/edit/delete posts with optional image or video attachments
Comments and reactions	Threaded comments and five reaction types per announcement
Direct messages	Private one-to-one messaging with unread counts
Family profiles	Per-member cards with name, age, and photo
Community search	Search all verified users by name
Playdates	Availability slot management and request/accept/decline flow
User profiles	Edit display name, city, state, and thumbnail

2. Goals and Non-Goals

Goals

- Provide a safe, verified community space limited to registered foster families.
- Keep the infrastructure simple enough to run on a single server without managed cloud services.
- Enable rapid iteration: local development requires only Node.js and a Gmail App Password.
- Cover all critical user flows with automated tests (unit, integration, and E2E).

Non-Goals

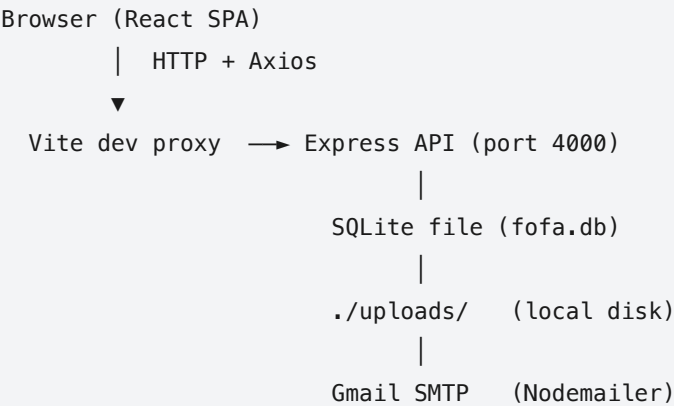
- Real-time features (WebSockets, push notifications) are not in scope for this version.
- Group messaging (more than two participants) is not implemented.
- The mobile app (`mobile/`) shares the same API but is treated as a separate delivery vehicle; it is not fully feature-parity with the web app in this baseline.
- Multi-tenancy or organization isolation — every verified user is in the same community.

3. System Architecture

3.1 Repository Layout

```
fofa/
├─ backend/      # Express 4 + TypeScript REST API (port 4000)
├─ frontend/     # React 18 + TypeScript SPA (port 5173)
├─ mobile/       # React Native / Expo app (Expo Go)
└─ cypress/      # End-to-end test suite
```

3.2 Runtime Topology



In production the Vite proxy is replaced by a reverse-proxy rule (e.g., Nginx) that forwards `/api` and `/uploads` to the Express process. The SPA's static build is served from the same Nginx instance.

3.3 Technology Choices

Concern	Choice	Rationale
API runtime	Node.js 18 + Express 4	Minimal footprint, large ecosystem
API language	TypeScript (strict)	Type safety across the full stack
Database	SQLite via <code>better-sqlite3</code>	Zero-dependency persistence; synchronous API simplifies controllers
Frontend framework	React 18	Mature, team familiarity
Frontend bundler	Vite 5	Fast HMR; built-in test runner integration

Concern	Choice	Rationale
Server state	TanStack Query v5	Declarative caching, pagination, and invalidation
Client state	Zustand	Tiny footprint; persists auth token to localStorage
Styling	Tailwind CSS 3 + CSS variables	Utility-first; dark/light mode via <code>class</code> strategy
Forms	React Hook Form	Uncontrolled inputs, minimal re-renders
HTTP client	Axios	Interceptors for JWT injection and 401 handling
Mobile	React Native / Expo	Code-share with web types and API layer

4. Data Model

All tables are created on startup via `runMigrations()` in `backend/src/utlis/migrate.ts` . The database runs in WAL mode with foreign keys enforced.

4.1 Entity Relationship Summary

```
users
├─ email_tokens          (1:many, on-delete cascade)
├─ password_reset_tokens (1:many, on-delete cascade)
├─ family_members        (1:many, on-delete cascade)
├─ announcements         (1:many, on-delete cascade)
│   └─ comments          (1:many, on-delete cascade)
│       └─ reactions      (1:many, on-delete cascade; unique per user per announcement)
├─ messages              (sender + receiver, both FK to users)
└─ availability_slots     (1:many, on-delete cascade)
    └─ playdate_requests  (1:many, on-delete cascade)
```

4.2 Table Definitions

`users`

Column	Type	Notes
id	TEXT PK	UUID v4

Column	Type	Notes
email	TEXT UNIQUE NOT NULL	Lowercase, normalized
password	TEXT NOT NULL	bcrypt, 12 rounds
name	TEXT NOT NULL	Display name
city	TEXT NOT NULL	
state	TEXT NOT NULL	
thumbnail	TEXT	Relative path under <code>/uploads/thumbnails/</code>
verified	INTEGER	0 = unverified, 1 = verified; login blocked until 1
created_at / updated_at	TEXT	ISO 8601 from <code>datetime('now')</code>

email_tokens

Column	Type	Notes
id	TEXT PK	UUID v4
user_id	TEXT FK → users	Cascade delete
token	TEXT UNIQUE NOT NULL	UUID v4 sent in verification link
expires_at	TEXT NOT NULL	24 hours after creation
used	INTEGER	Single-use; set to 1 on consumption

password_reset_tokens

Same shape as `email_tokens` with a 1-hour expiry.

family_members

Column	Type	Notes
id	TEXT PK	UUID v4
user_id	TEXT FK → users	
name	TEXT NOT NULL	
age	INTEGER NOT NULL	0–120 validated at API
thumbnail	TEXT	/uploads/thumbnails/

announcements

Column	Type	Notes
id	TEXT PK	UUID v4
user_id	TEXT FK → users	
content	TEXT NOT NULL	
media_url	TEXT	/uploads/media/<uuid>.<ext>
media_type	TEXT	'image' or 'video' ; CHECK constraint

comments

Column	Type	Notes
id	TEXT PK	UUID v4
announcement_id	TEXT FK → announcements	
user_id	TEXT FK → users	
content	TEXT NOT NULL	

reactions

Column	Type	Notes
id	TEXT PK	UUID v4

Column	Type	Notes
announcement_id	TEXT FK → announcements	
user_id	TEXT FK → users	
type	TEXT	CHECK IN ('like','love','hug','celebrate','support')
—	—	UNIQUE (announcement_id, user_id) — one reaction per user

messages

Column	Type	Notes
id	TEXT PK	UUID v4
sender_id	TEXT FK → users	
receiver_id	TEXT FK → users	
content	TEXT NOT NULL	
read	INTEGER	0 = unread, 1 = read; marked when recipient fetches thread

availability_slots

Column	Type	Notes
id	TEXT PK	UUID v4
user_id	TEXT FK → users	
date	TEXT NOT NULL	YYYY-MM-DD
start_time / end_time	TEXT NOT NULL	HH:MM
status	TEXT	CHECK IN ('free','busy') DEFAULT 'free'
note	TEXT	Optional free-text

playdate_requests

Column	Type	Notes
id	TEXT PK	UUID v4
requester_id	TEXT FK → users	
owner_id	TEXT FK → users	Slot owner
slot_id	TEXT FK → availability_slots	
message	TEXT	Optional context from requester
status	TEXT	CHECK IN ('pending','accepted','declined') DEFAULT 'pending'

4.3 Indexes

```

idx_announcements_user      ON announcements(user_id)
idx_announcements_created   ON announcements(created_at DESC)
idx_comments_announcement    ON comments(announcement_id)
idx_reactions_announcement   ON reactions(announcement_id)
idx_messages_sender         ON messages(sender_id)
idx_messages_receiver       ON messages(receiver_id)
idx_family_user             ON family_members(user_id)
idx_slots_user_date         ON availability_slots(user_id, date)
idx_requests_requester      ON playdate_requests(requester_id)
idx_requests_owner          ON playdate_requests(owner_id)

```

5. API Design

Base path: `/api`

All protected routes require `Authorization: Bearer <jwt>` header.

5.1 Auth (`/api/auth`)

Method	Path	Auth	Description
POST	<code>/register</code>	No	Create account; sends verification email

Method	Path	Auth	Description
GET	/verify-email?token=	No	Consume email token, set verified=1
POST	/login	No	Returns JWT + user object
POST	/forgot-password	No	Sends reset link (always 200 to prevent enumeration)
POST	/reset-password	No	Consumes reset token, updates password
POST	/change-password	Yes	Validates current password before update

Registration input validation (express-validator): email format, password min 8 chars, name/city/state non-empty.

5.2 Users (/api/users)

Method	Path	Auth	Description
GET	/me	Yes	Returns current user's profile
PUT	/me	Yes	Update profile fields + optional thumbnail upload
GET	/search?q=	Yes	Search users by name (partial match)
GET	/:id	Yes	Get any user's public profile

5.3 Family (/api/family)

Method	Path	Auth	Description
GET	/	Yes	List all family members for current user
POST	/	Yes	Add a family member (multipart with optional thumbnail)
PUT	/:id	Yes	Update a member
DELETE	/:id	Yes	Remove a member

5.4 Announcements (/api/announcements)

Method	Path	Auth	Description
GET	/	Yes	Paginated list (20/page, newest first) with reactions + comment count
POST	/	Yes	Create with optional media attachment (up to 100 MB)
PUT	/:id	Yes	Update content (own announcements only)
DELETE	/:id	Yes	Delete (own announcements only)
GET	/:id/comments	Yes	List all comments for an announcement
POST	/:id/comments	Yes	Add a comment
DELETE	/:id/comments/:commentId	Yes	Delete own comment
POST	/:id/reactions	Yes	Toggle reaction (add/change/remove)

Reaction toggle behavior: if the user has no reaction, it is added. If the same type is sent again, it is removed. If a different type is sent, the existing reaction is updated.

5.5 Messages (/api/messages)

Method	Path	Auth	Description
GET	/	Yes	List conversations (latest message per partner)
GET	/:partnerId	Yes	Get paginated thread; marks messages as read
POST	/	Yes	Send a message
GET	/unread/count	Yes	Count unread messages for navbar badge

Note on route ordering: /unread/count must be registered before /:partnerId in the router to prevent Express from matching "unread" as a partner ID. The current routes/index.ts registers conversations and messages before the unread route — this is a known ordering issue that works in practice because the unread endpoint is declared after /:partnerId but is matched correctly due to Express's exact string matching on "unread".

5.6 Playdates (/api/playdates)

Method	Path	Auth	Description
GET	<code>/availability/:userId</code>	Yes	Get slots; own view shows all statuses, others see only 'free'
POST	<code>/availability</code>	Yes	Add a slot
PUT	<code>/availability/:id</code>	Yes	Update own slot
DELETE	<code>/availability/:id</code>	Yes	Delete own slot
GET	<code>/requests</code>	Yes	All requests where current user is requester or owner
POST	<code>/requests</code>	Yes	Request a free slot (duplicate pending blocked)
PUT	<code>/requests/:id/respond</code>	Yes	Accept or decline (owner only)

5.7 Response Shapes

Pagination envelope (announcements):

```
{
  "data": [...],
  "pagination": { "page": 1, "limit": 20, "total": 42, "pages": 3 }
}
```

Error responses follow `{ "error": "<message>" }` with appropriate HTTP status codes (400, 401, 403, 404, 409, 413, 500).

6. Frontend Architecture

6.1 Application Shell

`App.tsx` wraps the entire tree in `QueryClientProvider` (TanStack Query) and `BrowserRouter`. Two layout wrappers handle route guarding:

- `GuestLayout` — redirects to `/dashboard` if a token is present.
- `ProtectedLayout` — redirects to `/login` if no token is present; renders the `Navbar`.

`QueryClient` is configured with `retry: 1` and `staleTime: 30,000 ms`.

6.2 Route Map

Path	Component	Protected
/login	LoginPage	No
/register	RegisterPage	No
/forgot-password	ForgotPasswordPage	No
/reset-password	ResetPasswordPage	No
/verify-email	VerifyEmailPage	No (public)
/dashboard	DashboardPage	Yes
/family	FamilyPage	Yes
/messages	MessagesPage	Yes
/community	CommunityPage	Yes
/profile	ProfilePage	Yes
/members/:id	MemberProfilePage	Yes
/playdates	PlaydatesPage	Yes
* / /	Redirect to /dashboard	—

6.3 State Management

State type	Tool	Location
Auth (user + token)	Zustand + localStorage persist	src/contexts/authStore.ts
Server data (queries)	TanStack Query	Inline useQuery / useMutation in page components
Local UI state	useState	Component level

The auth store is accessed outside React via `useAuthStore.getState()` in the Axios interceptor, avoiding circular dependency with React hooks.

6.4 API Layer

`src/services/api.ts` creates an Axios instance with:

- `baseUrl` from `VITE_API_URL` (defaults to `http://localhost:4000/api`).
- Request interceptor: injects `Authorization: Bearer <token>` from the Zustand store.
- Response interceptor: on 401 (excluding `/auth/` endpoints), clears the store and redirects to `/login`.

All API calls are wrapped in typed service modules exported from `src/services/index.ts`:

Service	Exported as
Auth	<code>authService</code>
Users	<code>userService</code>
Family	<code>familyService</code>
Announcements	<code>announcementService</code>
Messages	<code>messageService</code>
Playdates	<code>playdateService</code>

6.5 Design System

Token	Value
Brand green	<code>#4d9463</code> (dark: <code>#3a7049</code>)
Accent gold	<code>#f0b24f</code> (dark: <code>#d4962b</code>)
Body font	Nunito
Heading font	Titan One
Dark mode strategy	Tailwind <code>class</code> — toggled by adding/removing <code>dark</code> class on <code><html></code>
Conditional classes	<code>cn()</code> helper (<code>clsx</code> + <code>tailwind-merge</code>)

Surface and background colours are CSS variables, enabling seamless dark/light mode switching without class duplication.

7. Authentication and Authorization

7.1 Registration and Email Verification

1. Client POSTs `{ email, password, name, city, state }` to `/api/auth/register`.
2. Server hashes the password with bcrypt (12 rounds) and stores the user with `verified = 0`.
3. A UUID token is inserted into `email_tokens` with a 24-hour expiry; a verification email is sent via Nodemailer.
4. Client GETs `/api/auth/verify-email?token=<uuid>`.
5. Server checks the token is unused and not expired, then sets `verified = 1` and marks the token `used = 1` in a transaction.

7.2 Login

1. Client POSTs `{ email, password }`.
2. Server fetches the user, compares password with bcrypt, checks `verified = 1`.
3. On success, returns a JWT signed with `JWT_SECRET`, expiry `JWT_EXPIRES_IN` (default `7d`).
4. Token payload: `{ userId: string }`.
5. Client stores token in Zustand (persisted to localStorage as `fofa-auth`).

7.3 Protected Route Middleware

`authenticate` middleware (`backend/src/middleware/auth.middleware.ts`) extracts the `Bearer` token, verifies it with `jsonwebtoken`, and attaches `userId` to `req`. Controllers read `req.userId` directly — no further authorization helper is needed for most routes.

Ownership checks (e.g., delete own announcement) are done inline in controllers with a `WHERE id = ? AND user_id = ?` SQL pattern or an explicit `403` response.

7.4 Password Reset

1. Client POSTs email to `/api/auth/forgot-password`.
2. Server always returns HTTP 200 (enumeration prevention).
3. If the user exists, a token is inserted into `password_reset_tokens` (1-hour expiry) and an email is sent.
4. Client POSTs `{ token, password }` to `/api/auth/reset-password`.
5. Token is verified (unused + not expired), password updated, token marked used — all in a transaction.

8. File Uploads and Media

8.1 Storage

Multer stores files to the local filesystem under `UPLOADS_DIR` (default `./uploads/`). Two subdirectories are auto-created:

Directory	Used for	Size limit
<code>uploads/thumbnails/</code>	User and family member photos	5 MB
<code>uploads/media/</code>	Announcement images and videos	100 MB

Files are renamed to `<uuid>.<original-ext>` to avoid collisions and path traversal risks. Uploaded filenames are stored in the database as relative paths (e.g., `/uploads/thumbnails/abc123.jpg`).

8.2 Serving

Express serves `uploads/` as static files via `express.static` with `cross-origin` resource policy (required for `<img>` tags in the SPA). Vite proxies `/uploads` to the backend during development.

8.3 Accepted Types

Upload point	Accepted extensions
Thumbnails	<code>.jpg</code> , <code>.jpeg</code> , <code>.png</code> , <code>.gif</code> , <code>.webp</code> , <code>.heic</code> , <code>.heif</code>
Announcement media	All thumbnail types plus <code>.mp4</code> , <code>.mov</code> , <code>.webm</code> , <code>.m4v</code> , <code>.avi</code> , <code>.mkv</code>

Invalid types return HTTP 400. Files over the size limit return HTTP 413.

9. Email Service

`backend/src/services/email.service.ts` uses Nodemailer with Gmail SMTP (App Password authentication). Two emails are sent:

Email	Trigger	Link expiry
Verification	POST <code>/auth/register</code>	24 hours
Password reset	POST <code>/auth/forgot-password</code>	1 hour

In `NODE_ENV=test`, the email functions log the URL to stdout and return without sending. This makes tests deterministic without mocking Nodemailer.

Required environment variables:

```
GMAIL_USER=your@gmail.com
GMAIL_APP_PASSWORD=your_16_char_app_password
FRONTEND_URL=http://localhost:5173 # used to build links in emails
```

10. Testing Strategy

10.1 Test Layers

Layer	Tool	Location	What it covers
Backend unit/integration	Jest + ts-jest	<code>backend/</code>	Controllers, middleware, auth logic
Frontend component	Vitest + React Testing Library	<code>frontend/src/**/*.test.tsx</code>	UI behavior from the user's perspective
End-to-end	Cypress 13	<code>cypress/e2e/</code>	Full user flows against live servers

10.2 Backend Tests

Run with `npm test` in the `backend/` directory. Tests use `NODE_ENV=test` which suppresses real email sends and HTTP logging.

10.3 Frontend Tests

Run with `npm test` in the `frontend/` directory (Vitest). Configuration in `vite.config.ts`:

```
test: {
  globals: true,
  environment: 'jsdom',
  setupFiles: './src/tests/setup.ts',
  coverage: { provider: 'v8', reporter: ['text', 'lcov'] },
}
```

src/tests/setup.ts imports @testing-library/jest-dom and mocks react-hot-toast .

Component test files are co-located with their components. Example coverage:

Component	Test file
AnnouncementCard	AnnouncementCard.test.tsx
CreateAnnouncementForm	CreateAnnouncementForm.test.tsx
CommentsSection	CommentsSection.test.tsx
Navbar	Navbar.test.tsx
DashboardPage	DashboardPage.test.tsx
LoginPage	LoginPage.test.tsx
MessagesPage	MessagesPage.test.tsx

10.4 End-to-End Tests

Cypress specs map one-to-one to use cases. Both servers must be running before Cypress executes.

File	Use case
01-registration.cy.ts	UC-01: User Registration
02-login.cy.ts	UC-02: Login
03-announcements.cy.ts	UC-03: Announcements feed
04-family.cy.ts	UC-04: Family management
05-messaging.cy.ts	UC-05: Direct messaging

File	Use case
06-profile.cy.ts	UC-06: Profile editing
07-community-navigation.cy.ts	UC-07: Community search
08-playdates.cy.ts	UC-08: Playdate scheduling
09-member-profile.cy.ts	UC-09: View member profiles

10.5 Running Tests

```
# Backend unit tests
cd backend && npm test

# Frontend unit tests with coverage
cd frontend && npm test

# E2E (both servers must be running)
cd frontend && npm run cypress:open # interactive
cd frontend && npm run cypress:run  # headless CI
```

11. Deployment

11.1 Prerequisites

Requirement	Version
Node.js	18 or later
npm	8 or later
Gmail account with App Password	—
A Linux/macOS server or VM	—

SQLite requires no separate database server. No managed cloud service (RDS, S3, SES, etc.) is needed.

11.2 Environment Variables

Create `backend/.env` from `backend/.env.example` :

```
PORT=4000
JWT_SECRET=<long random string, min 32 chars>
JWT_EXPIRES_IN=7d
GMAIL_USER=your@gmail.com
GMAIL_APP_PASSWORD=<16-char app password>
FRONTEND_URL=https://your-domain.com
UPLOADS_DIR=./uploads
DB_PATH=./fofa.db
NODE_ENV=production
```

Create `frontend/.env` :

```
VITE_API_URL=https://your-domain.com/api
```

11.3 Build

```
# Backend (compile TypeScript)
cd backend
npm install --production=false
npm run build      # emits to dist/

# Frontend (Vite production build)
cd frontend
npm install
npm run build      # emits to dist/
```

The frontend build output in `frontend/dist/` is a static file tree that can be served from any web server.

11.4 Running in Production

```
# Run database migrations (idempotent – safe to re-run)
cd backend && node dist/utils/migrate.js

# Start API server
cd backend && node dist/index.js
```

The process should be managed by a process supervisor such as `pm2` or a systemd service unit to handle restarts on crash or server reboot.

Example `pm2` setup:

```
pm2 start backend/dist/index.js --name fofa-api
pm2 save
pm2 startup
```

11.5 Nginx Reverse Proxy

A typical Nginx configuration for a single server deployment:

```
server {
    listen 80;
    server_name your-domain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name your-domain.com;

    # TLS certificates (e.g., from Let's Encrypt)
    ssl_certificate      /etc/letsencrypt/live/your-domain.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/your-domain.com/privkey.pem;

    # Serve React SPA
    root /var/www/fofa/frontend/dist;
    index index.html;
    try_files $uri $uri/ /index.html;

    # Proxy API calls
    location /api/ {
        proxy_pass http://localhost:4000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # Proxy uploaded files
    location /uploads/ {
        proxy_pass http://localhost:4000;
    }
}
```

```
}  
  
}
```

11.6 File Persistence

Uploaded files are written to `UPLOADS_DIR` on the local disk. In a deployment with ephemeral storage (containers, some PaaS providers), this directory must be mounted to a persistent volume. The SQLite database file (`DB_PATH`) must similarly be on persistent storage.

11.7 Database Backups

Since SQLite is a single file, backups can be taken with:

```
# Online backup (safe while server is running, uses WAL checkpoint)  
sqlite3 /path/to/fofa.db ".backup /backups/fofa-$(date +%Y%m%d).db"
```

This can be run as a cron job. The backup file is a fully self-contained copy of the database.

11.8 Upgrade Procedure

- 1. Pull the latest code.
- 2. Install dependencies: `npm install` in `backend/` and `frontend/` .
- 3. Build: `npm run build` in both directories.
- 4. Run migrations: `node backend/dist/utils/migrate.js` (migrations are additive `CREATE TABLE IF NOT EXISTS` — safe to re-run against an existing database).
- 5. Restart the API process: `pm2 restart fofa-api` .
- 6. Copy the new frontend build to the web root.

12. Security Considerations

Area	Measure
Passwords	bcrypt with 12 rounds
JWT	Signed with <code>JWT_SECRET</code> ; 7-day expiry; no refresh token in this version
Transport	HTTPS enforced at Nginx; HTTP → HTTPS redirect

Area	Measure
HTTP headers	Helmet sets <code>Content-Security-Policy</code> , <code>X-Frame-Options</code> , <code>X-Content-Type-Options</code> , HSTS, etc.
Rate limiting	<code>express-rate-limit</code> : 200 requests per 15-minute window per IP
CORS	Restricted to <code>FRONTEND_URL</code> ; <code>credentials: true</code>
Account enumeration	<code>forgotPassword</code> always returns HTTP 200 regardless of whether the email exists
File uploads	Extension allowlist; size limits (5 MB thumbnails, 100 MB media); UUID filenames prevent path traversal
Input validation	<code>express-validator</code> on all mutation routes; validated before controllers run
SQL injection	All queries use <code>better-sqlite3</code> parameterized statements (no string interpolation)
Token single-use	Both email verification and password reset tokens have <code>used</code> flag; consumed atomically in a transaction

13. Known Limitations and Future Work

Item	Notes
No real-time messaging	Messages require a page refresh or periodic poll to surface new content. WebSockets or SSE would be the next step.
Local file storage	Media uploaded to disk is not replicated. Moving to object storage (S3-compatible) would enable horizontal scaling and CDN delivery.
Single SQLite instance	SQLite does not support multiple concurrent writers across processes. Acceptable for a single-node deployment; PostgreSQL would be needed for horizontal scaling.
No refresh token	JWT tokens are long-lived (7 days). Adding a short-lived access token + refresh token rotation would improve security.
Mobile feature gap	The React Native app (<code>mobile/</code>) shares the API but does not implement all web features (playdates, community search).

Item	Notes
No admin interface	Moderation tools (hide posts, ban users) do not exist. Community management relies on manual database access.
Email provider coupling	Nodemailer is hardcoded to Gmail SMTP. Abstracting to a transactional email provider (SendGrid, Resend) would improve deliverability.
No image thumbnailing	Profile photos and announcement images are stored at full uploaded size. Resizing on upload would reduce storage and bandwidth costs.

FoFa — Technical Design Document

Audience: Whole cross-functional team (product, engineering, design, QA) **Last updated:** 2026-04-15

Table of Contents

1. [Product Overview](#)
 2. [Tech Stack](#)
 3. [Architecture](#)
 4. [Database Schema](#)
 5. [API Specification](#)
 6. [Frontend Component Structure](#)
 7. [Testing Strategy](#)
 8. [Accessibility Strategy](#)
 9. [Performance & Scalability](#)
 10. [Responsiveness Strategy](#)
 11. [Deployment](#)
-

1. Product Overview

FoFa (Foster Families) is a private community platform for foster families to connect, share updates, and coordinate playdates.

Problem

Foster families are often isolated. There is no dedicated space for them to share announcements, find other families nearby, communicate privately, or schedule child-friendly meetups. FoFa fills that gap.

Who It's For

Foster parents and their household members who want to build community with other local foster families.

Core Features

Feature	Description
Announcements Feed	Post text/media updates; comment and react with 5 emotion types
Direct Messaging	Private 1-on-1 conversations with unread count tracking
Family Profiles	Add/edit family member cards with photos and ages
Community Search	Find other verified foster families by name
Playdates	Post availability slots; send/accept/decline playdate requests
Auth	Email-verified registration, JWT sessions, password reset via email
Dark / Light Mode	User-toggled color scheme persisted to localStorage

What FoFa Does NOT Do

- Group messaging or channels
- Real-time push notifications (polling only)
- Public or anonymous access — every page requires a verified account
- Native mobile apps (the `mobile/` directory is a separate, out-of-scope concern)
- Payment processing or subscription management
- Content moderation tooling

Platform Scope

Web SPA (React) + REST API (Node.js). Mobile is listed in the monorepo but is outside this document's scope.

2. Tech Stack

Backend

Layer	Technology	Version	Purpose
Runtime	Node.js	LTS	JavaScript server runtime
Framework	Express	^4.18.3	HTTP routing and middleware
Language	TypeScript	^5.4.3	Static typing
Database	SQLite (better-sqlite3)	^12.8.0	Embedded relational DB, WAL mode
Auth tokens	jsonwebtoken	^9.0.2	JWT signing/verification (7-day expiry)
Password hashing	bcryptjs	^2.4.3	bcrypt with 12 rounds
Email	Nodemailer	^8.0.3	Gmail SMTP for verification + password reset
File uploads	Multer	^1.4.5-lts.1	Disk storage for thumbnails and media
Input validation	express-validator	^7.0.1	Request body/query validation
Security	Helmet	^7.1.0	HTTP security headers
Rate limiting	express-rate-limit	^7.2.0	200 req/15 min per IP
Compression	compression	^1.7.4	Gzip response compression
Logging	morgan	^1.10.0	HTTP request logging (dev mode)
Testing	Jest + ts-jest	^29.7.0 / ^29.1.2	Unit and integration tests

Frontend

Layer	Technology	Version	Purpose
UI library	React	^18.2.0	Component-based UI
Language	TypeScript	^5.4.3	Static typing
Bundler	Vite	^5.2.6	Dev server and production builds

Layer	Technology	Version	Purpose
Routing	React Router v6	^6.22.3	Client-side navigation
Server state	TanStack Query	^5.28.4	Data fetching, caching, and background refetch
Client state	Zustand	^4.5.2	Auth and theme stores (persisted to localStorage)
HTTP client	Axios	^1.6.8	API calls with JWT interceptor
Forms	React Hook Form	^7.51.1	Controlled forms without re-render overhead
Styling	Tailwind CSS	^3.4.19	Utility-first CSS
Toasts	react-hot-toast	^2.4.1	Non-blocking user feedback
Date utils	date-fns	^3.6.0	Date formatting and arithmetic
File drop	react-dropzone	^14.2.3	Drag-and-drop file upload UI
Unit tests	Vitest + RTL	^1.4.0 / ^15.0.2	Component and hook tests
E2E tests	Cypress	^13.7.3	Full browser automation

Custom Theme Tokens

Token	Value	Tailwind Class
Brand green	#4d9463	brand / brand-dark / brand-light
Accent gold	#f0b24f	accent / accent-dark
Body font	Nunito	font-body
Heading font	Titan One	font-heading

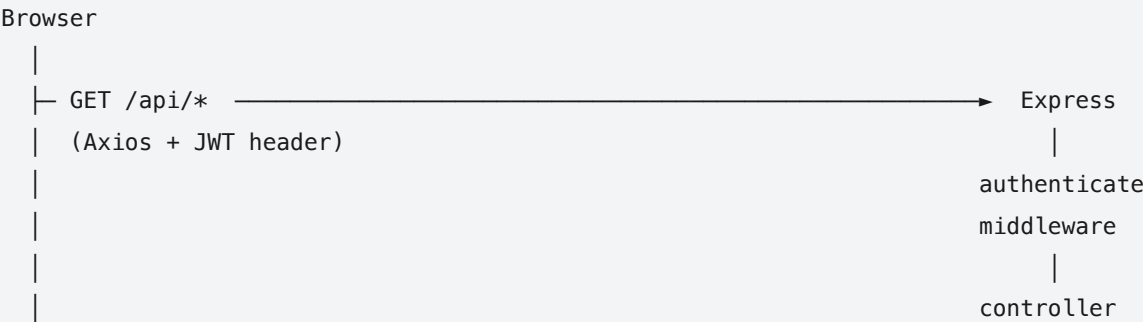
Dark mode is driven by a `dark` class on `<html>` and toggled via the `useThemeStore` Zustand store.

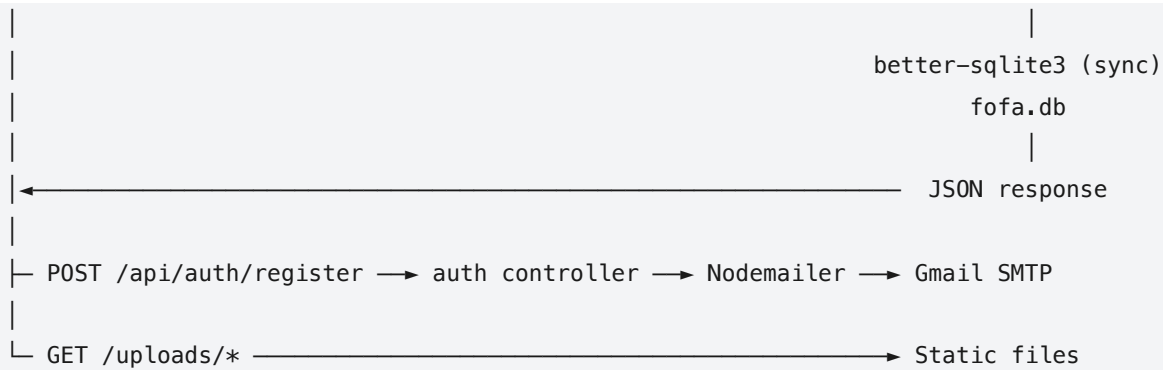
3. Architecture

Monorepo Structure

```
fofa/
├── backend/                # Express REST API
│   ├── src/
│   │   ├── controllers/   # Business logic per domain
│   │   ├── routes/        # Route definitions (single index.ts)
│   │   ├── middleware/    # auth, validate, upload
│   │   ├── services/      # email.service.ts
│   │   └── utils/
│   │       ├── db.ts      # SQLite singleton (WAL, FK ON)
│   │       └── migrate.ts # Schema migrations on startup
│   ├── tests/             # Jest test files
│   └── uploads/           # Multer upload destination (gitignored)
│       ├── thumbnails/
│       └── media/
├── frontend/              # React SPA
│   ├── src/
│   │   ├── components/    # Shared/reusable UI
│   │   ├── contexts/      # Zustand stores (auth, theme)
│   │   ├── hooks/         # Global reusable hooks
│   │   ├── pages/         # Route-level components
│   │   ├── services/      # Axios wrappers
│   │   └── types/         # Shared TypeScript interfaces
│   └── public/
├── cypress/               # E2E specs
│   ├── e2e/               # Test files (01-09)
│   ├── fixtures/
│   └── support/
└── mobile/                # React Native (out of scope)
```

Request Flow





Auth Architecture

1. **Register** — bcrypt-hashed password stored; UUID email token created and emailed (24 h TTL).
2. **Verify email** — token checked for `used = 0` and `expires_at` ; user marked `verified = 1` .
3. **Login** — credentials verified; JWT signed with `JWT_SECRET` (7-day expiry) returned to client.
4. **Authenticated requests** — `authenticate` middleware extracts `userId` from `Authorization: Bearer <token>` header, attaches it to `req.userId` .
5. **Password reset** — similar token flow; account enumeration prevented (identical response whether email exists or not).
6. **Token storage** — JWT stored in Zustand `fofa-auth` key in `localStorage` ; auto-injected by Axios request interceptor. On 401 (non-auth endpoint), Axios response interceptor calls `logout()` and redirects to `/login` .

4. Database Schema

ER Overview

```

users —< email_tokens
      —< password_reset_tokens
      —< family_members
      —< announcements —< comments
                        —< reactions
      —< messages (sender / receiver)
      —< availability_slots —< playdate_requests (requester / owner)
  
```

All primary keys are UUIDs (TEXT). Timestamps are stored as ISO-8601 TEXT (SQLite `datetime('now')`).

users

Column	Type	Constraints
id	TEXT	PRIMARY KEY
email	TEXT	UNIQUE NOT NULL
password	TEXT	NOT NULL (bcrypt hash)
name	TEXT	NOT NULL
city	TEXT	NOT NULL
state	TEXT	NOT NULL
thumbnail	TEXT	nullable (relative path under <code>/uploads</code>)
verified	INTEGER	NOT NULL DEFAULT 0 (0 = unverified, 1 = verified)
created_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>
updated_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

email_tokens

Column	Type	Constraints
id	TEXT	PRIMARY KEY
user_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
token	TEXT	UNIQUE NOT NULL
expires_at	TEXT	NOT NULL
used	INTEGER	NOT NULL DEFAULT 0
created_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

password_reset_tokens

Column	Type	Constraints
id	TEXT	PRIMARY KEY
user_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
token	TEXT	UNIQUE NOT NULL
expires_at	TEXT	NOT NULL
used	INTEGER	NOT NULL DEFAULT 0
created_at	TEXT	NOT NULL DEFAULT datetime('now')

family_members

Column	Type	Constraints
id	TEXT	PRIMARY KEY
user_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
name	TEXT	NOT NULL
age	INTEGER	NOT NULL (0–120)
thumbnail	TEXT	nullable
created_at	TEXT	NOT NULL DEFAULT datetime('now')
updated_at	TEXT	NOT NULL DEFAULT datetime('now')

announcements

Column	Type	Constraints
id	TEXT	PRIMARY KEY

Column	Type	Constraints
user_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
content	TEXT	NOT NULL
media_url	TEXT	nullable
media_type	TEXT	CHECK IN ('image', 'video', NULL)
created_at	TEXT	NOT NULL DEFAULT datetime('now')
updated_at	TEXT	NOT NULL DEFAULT datetime('now')

comments

Column	Type	Constraints
id	TEXT	PRIMARY KEY
announcement_id	TEXT	NOT NULL REFERENCES announcements(id) ON DELETE CASCADE
user_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
content	TEXT	NOT NULL
created_at	TEXT	NOT NULL DEFAULT datetime('now')
updated_at	TEXT	NOT NULL DEFAULT datetime('now')

reactions

Column	Type	Constraints
id	TEXT	PRIMARY KEY
announcement_id	TEXT	NOT NULL REFERENCES announcements(id) ON DELETE CASCADE
user_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE

Column	Type	Constraints
type	TEXT	NOT NULL CHECK IN ('like','love','hug','celebrate','support')
created_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>
—	—	UNIQUE(announcement_id, user_id) — one reaction per user per post

messages

Column	Type	Constraints
id	TEXT	PRIMARY KEY
sender_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
receiver_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
content	TEXT	NOT NULL
read	INTEGER	NOT NULL DEFAULT 0
created_at	TEXT	NOT NULL DEFAULT <code>datetime('now')</code>

availability_slots

Column	Type	Constraints
id	TEXT	PRIMARY KEY
user_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
date	TEXT	NOT NULL (YYYY-MM-DD)
start_time	TEXT	NOT NULL (HH:MM)
end_time	TEXT	NOT NULL (HH:MM)
status	TEXT	NOT NULL CHECK IN ('free','busy') DEFAULT 'free'
note	TEXT	nullable

Column	Type	Constraints
created_at	TEXT	NOT NULL DEFAULT datetime('now')

playdate_requests

Column	Type	Constraints
id	TEXT	PRIMARY KEY
requester_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
owner_id	TEXT	NOT NULL REFERENCES users(id) ON DELETE CASCADE
slot_id	TEXT	NOT NULL REFERENCES availability_slots(id) ON DELETE CASCADE
message	TEXT	nullable
status	TEXT	NOT NULL CHECK IN ('pending','accepted','declined') DEFAULT 'pending'
created_at	TEXT	NOT NULL DEFAULT datetime('now')
updated_at	TEXT	NOT NULL DEFAULT datetime('now')

Indexes Summary

Index Name	Table	Columns	Purpose
idx_announcements_user	announcements	user_id	Filter by author
idx_announcements_created	announcements	created_at DESC	Feed ordering
idx_comments_announcement	comments	announcement_id	Load comments per post
idx_reactions_announcement	reactions	announcement_id	Aggregate reactions per post
idx_messages_sender	messages	sender_id	Sent message lookup

Index Name	Table	Columns	Purpose
idx_messages_receiver	messages	receiver_id	Inbox lookup
idx_family_user	family_members	user_id	Load family per user
idx_slots_user_date	availability_slots	(user_id, date)	Calendar queries
idx_requests_requester	playdate_requests	requester_id	Sent requests
idx_requests_owner	playdate_requests	owner_id	Received requests

5. API Specification

Base URL: `http://localhost:4000/api` (dev) / configured via `VITE_API_URL` (prod)

Auth method: `Authorization: Bearer <JWT>` header on all protected routes.

File upload content type: `multipart/form-data` for any endpoint that accepts a `thumbnail` or `media` field.

Auth

Method	Path	Auth	Description
POST	<code>/auth/register</code>	No	Create account; sends verification email
GET	<code>/auth/verify-email?token=</code>	No	Mark email as verified (single-use token)
POST	<code>/auth/login</code>	No	Returns <code>{ token, user }</code> on success
POST	<code>/auth/forgot-password</code>	No	Sends reset email (enumeration-safe)
POST	<code>/auth/reset-password</code>	No	Resets password using emailed token
POST	<code>/auth/change-password</code>	Yes	Changes password for logged-in user

Users

Method	Path	Auth	Description
GET	/users/me	Yes	Returns the current user's profile
PUT	/users/me	Yes	Update profile; accepts multipart/form-data with optional thumbnail
GET	/users/search?q=	Yes	Search users by name (partial match)
GET	/users/:id	Yes	Get any user profile by ID

Family Members

Method	Path	Auth	Description
GET	/family	Yes	List caller's family members
POST	/family	Yes	Add a family member (multipart/form-data , requires name and age)
PUT	/family/:id	Yes	Update a family member (optional thumbnail)
DELETE	/family/:id	Yes	Remove a family member

Announcements

Method	Path	Auth	Description
GET	/announcements?page=	Yes	Paginated feed (newest first); responses include author, comment count, reactions
POST	/announcements	Yes	Create post (multipart/form-data , optional media file)
PUT	/announcements/:id	Yes	Update post content (own posts only)
DELETE	/announcements/:id	Yes	Delete post (own posts only)

Method	Path	Auth	Description
GET	/announcements/:id/comments	Yes	List comments on a post
POST	/announcements/:id/comments	Yes	Add a comment
DELETE	/announcements/:id/comments/:commentId	Yes	Delete a comment
POST	/announcements/:id/reactions	Yes	Toggle a reaction (creates if absent, deletes if present)

Reaction types: like , love , hug , celebrate , support . One reaction per user per announcement (UNIQUE constraint). Sending the same type again removes it.

Messages

Method	Path	Auth	Description
GET	/messages	Yes	List all conversations (latest message per partner)
GET	/messages/:partnerId?page=	Yes	Paginated message thread; marks messages as read automatically
POST	/messages	Yes	Send a message (receiverId , content)
GET	/messages/unread/count	Yes	Returns { count: number } of unread messages

Note: Fetching a thread (GET /messages/:partnerId) marks all messages in that thread as read = 1 server-side. The Navbar polls /messages/unread/count every 30 seconds.

Playdates

Method	Path	Auth	Description
GET	/playdates/availability/:userId	Yes	Get availability slots for any user
POST	/playdates/availability	Yes	Add an availability slot (date , start_time , end_time , optional status , note)

Method	Path	Auth	Description
PUT	/playdates/availability/:id	Yes	Update own slot
DELETE	/playdates/availability/:id	Yes	Delete own slot
GET	/playdates/requests	Yes	List all requests (sent and received)
POST	/playdates/requests	Yes	Request a playdate on a specific slot (slotId , optional message)
PUT	/playdates/requests/:id/respond	Yes	Accept or decline a request (status : accepted or declined)

Health

Method	Path	Auth	Description
GET	/health	No	Returns { status: "ok", env }

6. Frontend Component Structure

Routing Table

Route	Page	Auth Required
/login	LoginPage	No (guest only)
/register	RegisterPage	No (guest only)
/forgot-password	ForgotPasswordPage	No (guest only)
/reset-password	ResetPasswordPage	No (guest only)
/verify-email	VerifyEmailPage	No
/dashboard	DashboardPage	Yes
/family	FamilyPage	Yes

Route	Page	Auth Required
/messages	MessagesPage	Yes
/community	CommunityPage	Yes
/profile	ProfilePage	Yes
/members/:id	MemberProfilePage	Yes
/playdates	PlaydatesPage	Yes
/*	—	Redirects to /dashboard

Route protection is implemented with two layout wrappers: `ProtectedLayout` (redirects to `/login` if no token) and `GuestLayout` (redirects to `/dashboard` if already authenticated).

Pages

Page	Responsibility
LoginPage	Email/password login form
RegisterPage	Registration form (email, password, name, city, state)
ForgotPasswordPage	Request password reset email
ResetPasswordPage	Set new password via emailed token
VerifyEmailPage	Handles email verification link redirect
DashboardPage	Announcements feed with pagination, create post, reactions, comments
FamilyPage	Manage own family members (add, edit, delete)
MessagesPage	Direct message inbox and conversation thread
CommunityPage	Search for other users
ProfilePage	Edit own profile (name, city, state, avatar) and change password
MemberProfilePage	View another user's profile and family members
PlaydatesPage	Calendar view, manage availability slots, send/respond to requests

Shared Components

components/ui/

Component	Purpose
Avatar	User avatar with fallback initials
Button	Styled button with variants (primary, secondary, danger)
Input	Labeled input with error display
Modal	Accessible overlay dialog
Logo	FoFa brand logo SVG

components/dashboard/

Component	Purpose
Navbar	Top navigation bar + mobile bottom tab bar; shows unread message and playdate badges
CommentsSection	Expandable comments list and add-comment form

components/announcements/

Component	Purpose
AnnouncementCard	Single feed post with reactions, comment count, media
CreateAnnouncementForm	Rich post creation with optional media upload

components/family/

Component	Purpose
FamilyMemberCard	Displays a family member with edit/delete actions

components/messages/

Component	Purpose
MessagesPanel	Two-pane message UI: conversation list + thread

components/playdates/

Component	Purpose
MonthCalendar	Month-grid view for availability
WeekCalendar	Week-column view for availability slots
TimePicker	Time input control for slot start/end

components/WithAuth

Legacy HOC wrapper — superseded by ProtectedLayout in App.tsx but remains available.

Where to Put a New Component

If the component is used in only one feature: put it inside that feature's page folder (e.g., pages/FamilyPage/). If it is shared across two or more features: put it in components/ under the appropriate subdirectory. Each component file should be PascalCase and contain exactly one exported component.

State Management Summary

State type	Tool	Store/location
Auth (user, JWT)	Zustand + persist	src/contexts/authStore.ts (key: fofa-auth)
Theme (dark/light)	Zustand + persist	src/contexts/themeStore.ts
Server data	TanStack Query	Inline useQuery / useMutation calls in pages/components
Local UI state	useState	Component-local

Server state is never duplicated into Zustand stores.

API Service Layer

All API calls go through `src/services/index.ts`, which re-exports domain-grouped service objects. Each call is typed against the interfaces in `src/types/index.ts`.

Service	Methods
<code>authService</code>	<code>register</code> , <code>login</code> , <code>verifyEmail</code> , <code>forgotPassword</code> , <code>resetPassword</code> , <code>changePassword</code>
<code>userService</code>	<code>getMe</code> , <code>updateProfile</code> , <code>search</code> , <code>getById</code>
<code>familyService</code>	<code>getAll</code> , <code>add</code> , <code>update</code> , <code>remove</code>
<code>announcementService</code>	<code>getAll</code> , <code>create</code> , <code>update</code> , <code>remove</code> , <code>getComments</code> , <code>addComment</code> , <code>deleteComment</code> , <code>toggleReaction</code>
<code>messageService</code>	<code>getConversations</code> , <code>getMessages</code> , <code>send</code> , <code>getUnreadCount</code>
<code>playdateService</code>	<code>getAvailability</code> , <code>addSlot</code> , <code>updateSlot</code> , <code>deleteSlot</code> , <code>getRequests</code> , <code>createRequest</code> , <code>respond</code>

7. Testing Strategy

Three-Layer Table

Layer	Tool	Location	What it tests
Backend unit/integration	Jest + ts-jest	<code>backend/tests/</code>	Controllers, middleware, auth logic
Frontend unit/component	Vitest + React Testing Library	<code>frontend/src/**/*.test.tsx</code> and <code>frontend/src/tests/unit/</code>	Component rendering, user interactions, form validation
End-to-end	Cypress	<code>cypress/e2e/</code>	Full user journeys through the browser

Backend (Jest)

- Tests run in a Node environment against real SQLite (in-memory or temp DB recommended per test suite).
- Use `ts-jest` for TypeScript compilation.
- Coverage collected from `src/**/*.ts` excluding entry point and migrate utility.

Frontend (Vitest + RTL)

- Tests run in jsdom. Setup file mocks `import.meta.env` and `react-hot-toast`.
- Use `getByRole`, `getByLabelText`, `getByText` — never query by class or test ID unless unavoidable.
- Mock at the network boundary using module mocks for the service layer (MSW recommended for future adoption).
- No snapshot tests.

End-to-End (Cypress)

Specs cover the full user journey in numbered order:

Spec	Scenario
<code>01-registration.cy.ts</code>	Register new account
<code>02-login.cy.ts</code>	Login, invalid credentials
<code>03-announcements.cy.ts</code>	Create post, comment, react
<code>04-family.cy.ts</code>	Add/edit/delete family member
<code>05-messaging.cy.ts</code>	Send and receive messages
<code>06-profile.cy.ts</code>	Edit profile, change password
<code>07-community-navigation.cy.ts</code>	Search users, view profiles
<code>08-playdates.cy.ts</code>	Add slot, send/respond to request
<code>09-member-profile.cy.ts</code>	View another user's profile

TDD Workflow

1. Write a failing test that describes the expected behavior.

2. Run tests to confirm failure.
3. Implement the minimum code to make the test pass.
4. Refactor while keeping tests green.

Test Commands

```
# Backend unit tests
cd backend && npm test

# Backend tests in watch mode
cd backend && npm run test:watch

# Frontend unit tests (single run + coverage)
cd frontend && npm test

# Frontend tests in watch mode
cd frontend && npm run test:watch

# Cypress interactive runner
cd frontend && npm run cypress:open

# Cypress headless CI run
cd frontend && npm run cypress:run
```

Coverage Targets

Layer	Target
Backend statements	≥ 80%
Frontend statements	≥ 70%
E2E critical paths	100% of core flows covered

8. Accessibility Strategy

Target: WCAG 2.1 Level AA

FoFa is used by foster families who may include users with disabilities. Accessibility is a core quality bar, not a post-launch concern.

Principles

Principle	Implementation
Perceivable	All images have alt text or <code>aria-label</code> ; color is never the only differentiator
Operable	All interactive elements reachable and usable via keyboard
Understandable	Labels and instructions are clear; errors are field-level and descriptive
Robust	Semantic HTML; ARIA roles only where native semantics are insufficient

Implementation Checklist

Semantic HTML

- ☐ Use `<nav>` for navigation bars (Navbar already uses `aria-label="Main navigation"`)
- ☐ Use `<main>` , `<section>` , `<article>` for page structure
- ☐ Headings follow a logical hierarchy (h1 → h2 → h3)

Forms

- ☐ Every input has an associated `<label>` (via `for / id` or `aria-labelledby`)
- ☐ Validation errors rendered next to the field, not only in a toast
- ☐ Required fields indicated with `aria-required` or visible marker

Interactive Components

- ☐ Buttons use `<button>` (not `<div onClick>`)
- ☐ Modals trap focus and return focus on close
- ☐ Dropdown menus use `role="menu"` / `role="menuitem"` with keyboard navigation
- ☐ Active nav links set `aria-current="page"`
- ☐ Profile menu button sets `aria-expanded` and `aria-haspopup="menu"`
- ☐ Badge counts surfaced to screen readers via `aria-label` on nav links (e.g., "Messages, 3 pending")

Color & Contrast

- ☐ Brand green (#4d9463) on white background meets AA for large text; verify with a contrast checker for small text
- ☐ Dark mode tokens maintain equivalent contrast ratios
- ☐ Focus styles are visible (Tailwind `focus-visible:ring` or equivalent)

Testing Approach

Type	Tool
Automated lint	<code>eslint-plugin-jsx-a11y</code> (recommended addition)
Manual keyboard test	Tab through every interactive element on each page
Screen reader test	VoiceOver (macOS) on key flows (login, post creation, messaging)
Cypress a11y	<code>cypress-axe</code> (recommended addition)

9. Performance & Scalability

Frontend Performance

Concern	Strategy
Bundle size	Vite tree-shaking + code splitting per route (React lazy/Suspense — recommended)
API re-fetches	TanStack Query <code>staleTime: 30_000</code> default; feed uses manual pagination
Image loading	Thumbnails are small (5 MB upload limit, stored as-is — server-side resize recommended)
Unread polling	Navbar polls unread count and playdate requests every 30 s (not push-based)
Dark/light flash	Theme class applied before paint (Zustand persist reads synchronously from localStorage)

Backend Performance

Concern	Strategy
Response compression	<code>compression</code> middleware (gzip) on all responses
DB query performance	SQLite indexes on all foreign keys and common filter columns

Concern	Strategy
DB write throughput	WAL journal mode enables concurrent reads during writes
File I/O	Multer disk storage; heavy media uploads limited to 100 MB
Request abuse	Rate limit: 200 requests per 15 minutes per IP
Body size	JSON body limited to 10 MB

Scalability Constraints

Constraint	Threshold	Migration Path
SQLite concurrency	Single-writer; suitable for hundreds of concurrent users	Migrate to PostgreSQL when write contention emerges
Local file storage	Disk-bound; limited by server storage	Migrate uploads to S3-compatible object storage
In-process Node.js	Single process, no clustering	Add <code>cluster</code> module or move to containerized autoscaling
JWT stateless auth	No server-side token invalidation	Add a token revocation list (Redis) if forced logout is required

Caching Strategy

Layer	Current State	Recommended Next Step
API responses	None server-side	Add Cache-Control headers for public static assets
React Query	30 s stale time	Tune per-query based on volatility
Static assets	Vite content-hash filenames	CDN in front of the production build
DB query results	None	Add LRU cache for expensive read queries (e.g., community search)

10. Responsiveness Strategy

Breakpoints

Tailwind default breakpoints are used:

Breakpoint	Min Width	Target Devices
(default)	0 px	Mobile phones (portrait)
sm	640 px	Mobile landscape
md	768 px	Tablets, small laptops
lg	1024 px	Laptops
xl	1280 px	Desktops

The content max-width is capped at 1100 px (max-w-[1100px]).

Layout Strategy Per Section

Section	Mobile	Tablet (md)	Desktop
Navigation	Bottom tab bar (fixed)	Top navbar	Top navbar
Feed cards	Full-width single column	Full-width single column	Centered, max 1100 px
Messages	Full-screen single pane (conversation list or thread)	Two-pane side-by-side	Two-pane side-by-side
Family members	Single-column card list	Two-column grid	Three-column grid
Profile form	Stacked fields	Stacked fields	Constrained width centered
Playdates calendar	Month view, scrollable	Week view	Week view

Implementation Rules

- Use Tailwind responsive prefixes (md: , lg:) — never write custom @media queries.
- Mobile layout is the default (no prefix); larger-screen overrides are additive.

- Bottom tab bar shown on `< md` , hidden on `md:` and above (`md:hidden` / `hidden md:flex`).
- Profile dropdown and username are hidden on mobile nav to save space.
- Interactive tap targets meet the 44 × 44 px minimum on mobile.

Typography Scaling

Element	Mobile	Desktop
Body text	<code>text-sm</code> (14 px)	<code>text-base</code> (16 px)
Card title	<code>text-base</code>	<code>text-lg</code>
Page headings	<code>text-xl</code>	<code>text-2xl</code>
Nav labels	<code>text-[0.68rem]</code>	<code>text-[0.92rem]</code>

11. Deployment

Environment Variables

Backend (`backend/.env`)

Variable	Required	Description
<code>DATABASE_URL</code> / <code>DB_PATH</code>	No (default <code>./fofa.db</code>)	Path to SQLite database file
<code>JWT_SECRET</code>	Yes	Secret for signing JWTs (min 32 chars recommended)
<code>EMAIL_USER</code>	Yes	Gmail address for Nodemailer SMTP
<code>EMAIL_PASS</code>	Yes	Gmail app password for SMTP
<code>FRONTEND_URL</code>	Yes	Allowed CORS origin (e.g., <code>https://fofa.example.com</code>)
<code>PORT</code>	No (default <code>4000</code>)	Port the API server listens on

Variable	Required	Description
<code>UPLOADS_DIR</code>	No (default <code>./uploads</code>)	Directory for Multer file storage

Frontend (`frontend/.env`)

Variable	Required	Description
<code>VITE_API_URL</code>	No (default <code>http://localhost:4000/api</code>)	Base URL for API calls

Build & Start

```
# Backend – compile TypeScript and start
cd backend
npm run build      # tsc → dist/
npm start          # node dist/index.js

# Frontend – build static assets
cd frontend
npm run build      # tsc + vite → dist/
npm run preview    # preview the production build locally
```

Database Migrations

Migrations run automatically on every server start via `runMigrations()` in `src/utils/migrate.ts` . All `CREATE TABLE IF NOT EXISTS` and `CREATE INDEX IF NOT EXISTS` statements are idempotent — safe to run repeatedly.

To run migrations manually:

```
cd backend && npx ts-node src/utils/migrate.ts
```

File Uploads

Uploaded files are stored on the local filesystem at `backend/uploads/` . In a production deployment:

- Mount a persistent volume at the `UPLOADS_DIR` path, or

- Migrate to S3-compatible object storage and update the upload middleware and static file serving accordingly.

Recommended Production Stack

Concern	Recommendation
Process manager	PM2 (<code>pm2 start dist/index.js</code>) or a container (Docker)
Reverse proxy	Nginx — serves frontend static files and proxies <code>/api</code> and <code>/uploads</code> to Node.js
TLS	Let's Encrypt via Certbot on the Nginx layer
DB persistence	SQLite file on a mounted volume; daily backup via <code>sqlite3 .backup</code> or <code>litestream</code>
Frontend hosting	Nginx static file serving or CDN (Cloudflare Pages, Vercel)
Email	Gmail SMTP with an App Password, or migrate to SendGrid/SES for higher volume

CI/CD Checklist

- ☐ `cd backend && npm test` passes
- ☐ `cd frontend && npm test` passes
- ☐ `cd frontend && npm run build` exits 0 (TypeScript strict check passes)
- ☐ Environment variables injected via CI secrets (never committed)
- ☐ Database backed up before each production deploy
- ☐ Cypress smoke suite (`npm run cypress:run`) passes against staging before promoting to production